

# GRAPHPTC: MODELING PROGRAMMATIC TOOL CALLING AS A DYNAMIC EXECUTION GRAPH

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Tool calling has become the primary way large language models act on external environments, and its form is shifting from issuing one tool call per model turn toward Programmatic Tool Calling (PTC), where a model writes a program that composes many tool calls, control flow, and intermediate data processing inside a single block executed in a persistent runtime. PTC reduces model-tool interaction rounds and expresses in-block control flow that a per-call protocol cannot, yet across program blocks it records execution as a linear history of code, standard output, and errors. Such a history leaves the internal state of an execution untrackable, so whether a write took hold, which earlier value a later step depends on, and whether two results carry the same content all have to be recovered from raw text. The cost is visible in accuracy, since PTC falls below plain per-call tool calling 14 times over the seven benchmark settings and five backbones we evaluate and loses 34.5 points on the worst one, so a programmatic runtime alone does not carry every setting. To this end, we propose GRAPHPTC, which preserves the programmatic expressiveness of PTC and maintains a task-level dynamic execution graph built from real running traces. Specifically, GRAPHPTC models the internal logic of each block as a typed subgraph of tool effects, artifacts, states, and failures, and models cross-block dependency as a multi-relation graph over which the agent declares an intent and chooses to continue, patch a failed step, replan a dependency path, or answer. Unlike a linear history that flattens these relations into strings, the graph keeps them addressable, so the agent reads what an execution changed instead of re-parsing what it printed. Experiments show that GRAPHPTC is the best method on every setting and every backbone, improving over Direct Tool Calling by 6.8 points and over PTC by 6.3 points while reading 1.7 to 2.3 times fewer input tokens.

## 1 INTRODUCTION

Tool calling is the primary interface through which large language models (LLMs) act on external environments, and its form has been shifting. Early agents issue a single tool call per model turn and read its result before deciding the next call, following the reasoning-and-acting paradigm (Yao et al., 2023; Schick et al., 2023; Qin et al., 2024). Recent systems instead adopt *Programmatic Tool Calling* (PTC), where the model writes a program that composes many tool calls with loops, conditionals, and intermediate data processing, and the program runs in a persistent runtime whose variables carry across turns (Gao et al., 2023; Wang et al., 2024; Hugging Face, 2025; Anthropic, 2025). By packing a coherent phase of work into one executable block, PTC reduces the number of model-tool interaction rounds, expresses control flow that a call-per-turn protocol cannot, and lets a later block reuse variables produced by an earlier one. However, the expressiveness that PTC gains inside a block is not matched by how it organizes work across blocks.

Across blocks, the history that a PTC agent conditions on is a flat sequence of code, standard output, and error text ordered by time, which is a lossy record of what execution did. A single block on the AppWorld benchmark (Trivedi et al., 2024) issues a sequence of tool calls whose read or write effects and produced values are flattened into one output string, and the hardest tasks chain more than fifteen such blocks. Once flattened, the internal state of an execution is no longer trackable, so whether a write took hold, which earlier value a new step depends on, and whether two differently phrased queries returned the same content all have to be recovered by re-reading raw text. The cost shows

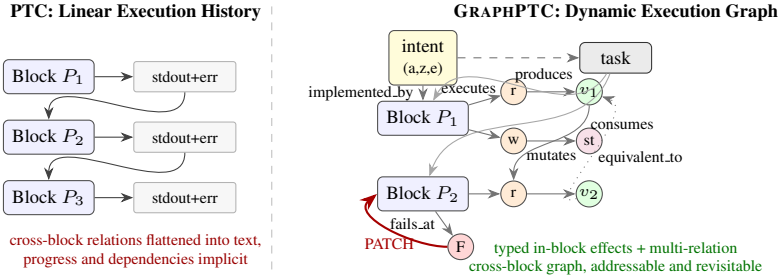


Figure 1: PTC records execution as a linear stream of code and output where cross-block dependencies and progress are lost, whereas GRAPHPTC maintains a dynamic execution graph that models the internal effects of each block (tool calls with read  $r$  or write  $w$  effects, artifacts  $v$ , states  $st$ , failures  $F$ ) and cross-block dependencies (consumes, equivalence, supersession), over which a declared intent selects continue, patch, replan, or answer.

up in accuracy, since PTC falls below plain per-call tool calling 14 times over the seven benchmark settings and five backbones we evaluate, and Claude-Opus-5 loses 34.5 points on AppWorld-Normal once it moves from per-call tool calling to PTC, so packing more work into a block does not by itself carry a setting.

What an execution induces is a set of relations, where a call yields an artifact, a later call consumes that artifact, and a write supersedes an earlier state version. A time-ordered transcript keeps only the order in which these relations appeared, so the obstacle is the form of the record and not its length, and a fix has to restore the structure that execution already carries. This concatenated representation produces three limitations that build on one another. At the representation layer, the history is a *linear execution context*, where each of the relations above survives only as text that the agent parses again, so it rebuilds prior results and their dependencies from scratch. Because that representation carries no unified form for an effect, the analysis layer sees only *implicit execution progress* and never relates what a block intended to achieve to what it actually changed, so a block that raises no error can still re-fetch an existing result without advancing the task. With progress left implicit, the decision layer acts on *unstructured decision feedback* that describes the current block alone, and an agent inferring from scattered history whether to proceed, fix the program, or change strategy easily repeats a prior attempt or extends an unproductive path.

To this end, we propose GRAPHPTC, which keeps the programmatic expressiveness of PTC and maintains a task-level *dynamic execution graph* built incrementally from real running traces. As shown in Figure 1, GRAPHPTC makes two structural changes to PTC. First, it models the internal logic of each block, so a code execution becomes a typed subgraph of tool calls with read or write effects, produced artifacts, state versions, and localized failures instead of an opaque output string. Second, it models cross-block dependency as a multi-relation graph instead of a linear text stream, linking equivalent results, reused artifacts, and superseded states so that history becomes addressable. These two changes support three stages that answer the three limitations in turn. Specifically, to replace the linear execution context, graph-based execution modeling projects each trace into a shared typed graph and links cross-block dependencies. To make execution progress explicit, graph-based progress assessment binds a declared intent to the graph change it produces and separates new results from repeated ones. To structure the decision feedback, graph-based action selection turns the graph into a bounded summary over which the agent chooses to continue, patch a failed step, replan a dependency path, or answer.

We evaluate GRAPHPTC on seven tool-use benchmark settings with five backbone LLMs. GRAPHPTC is the best method on every setting and every backbone, improving over Direct Tool Calling by 6.8 points and over PTC by 6.3 points on the seven-setting average. It is also cheaper to run, reading 1.7 to 2.3 times fewer input tokens than Direct Tool Calling on every benchmark, cutting model rounds by 54 to 64% on AutomationBench, and reaching the accuracy of Direct Tool Calling with about 2 times fewer rounds on the AppWorld frontier. Our contributions are three-fold.

- **Structural Limitation of Programmatic Tool Calling.** We show that PTC is not a uniform upgrade over per-call tool calling, falling below it 14 times over seven benchmark settings and

five backbones, and we trace this to a linear execution history in which tool effects, artifacts, and state versions are untrackable.

- **Graph-Structured Tool Calling.** To the best of our knowledge, GRAPHPTC is the first framework that models programmatic tool calling as a task-level dynamic execution graph, and it binds a declared intent to the graph change it produces, so the agent judges progress from observed effects and treats recovery as an addressable graph operation over continue, patch, replan, and answer.
- **Strong Performance.** GRAPHPTC is the best method on every one of the seven settings for all five backbones, improving over Direct Tool Calling by 6.8 points and over PTC by 6.3 points while reading fewer input tokens and using fewer interaction rounds.

## 2 RELATED WORK

**Tool Calling and Programmatic Agents.** Tool-augmented LLMs have progressed from single-call protocols to programmatic execution. Early work teaches models to emit one structured tool call per turn and interleave it with reasoning, as in Toolformer (Schick et al., 2023), ReAct (Yao et al., 2023), ToolLLM (Qin et al., 2024), and Gorilla (Patil et al., 2024), and multi-agent frameworks such as AutoGen (Wu et al., 2023) and MetaGPT (Hong et al., 2024) coordinate several such agents. A parallel line expresses actions as code, starting from program-aided reasoning (Gao et al., 2023; Chen et al., 2023) and maturing into code-as-action agents that emit executable programs calling tools in a live interpreter, including CodeAct (Wang et al., 2024) and code-agent libraries (Hugging Face, 2025).

**Enhancement of Tool Calling.** Direct tool calling has been improved along two directions. One direction enriches the history the agent reads, where reflection turns past outcomes into natural-language advice (Shinn et al., 2023; Madaan et al., 2023), memory systems extend the usable context (Packer et al., 2023; Rasmussen et al., 2025; Xu et al., 2025; Summers et al., 2024), and provenance models in data management record how each result was derived (Davidson & Freire, 2008; Herschel et al., 2017). These additions index facts and entities for later retrieval and leave the execution itself as text. The other direction changes how actions are issued, reducing the one-call-per-turn cost by batching dependent calls into a single program, and it arrives at programmatic tool calling, which runs a model-written program against many tools in one runtime (Anthropic, 2025).

PTC buys expressiveness and fewer interaction rounds inside a block, but it still hands the model a linear transcript of code and output across blocks, so tool effects, artifacts, and state versions are untrackable and the dependency structure that execution induces is left for the model to reconstruct. This is why PTC falls below direct tool calling 14 times over seven benchmark settings and five backbones. We propose GRAPHPTC to remove this bottleneck, maintaining a task-level dynamic execution graph that models in-block tool effects and cross-block dependencies and driving action selection from a bounded summary of that graph, so the agent keeps the efficiency of programmatic execution and recovers the reliability that a linear history loses.

## 3 GRAPHPTC

GRAPHPTC retains the ability of PTC to organize many tool calls and intermediate data processing into one executable program, and augments it with a task-level dynamic execution graph that is built from real running traces. The design follows three stages that respond one to one to the three limitations of Section 1.

We first fix notation. Let  $P_t$  be the program the agent writes at round  $t$ ,  $\mathcal{G}_{t-1}$  the accumulated graph before this round, and  $\tau_t$  the trace obtained by executing  $P_t$  with instrumented tools. When the agent writes  $P_t$  it also declares an intent  $I_t$ . The runtime projects the trace into the graph, extracts the change, and produces a bounded feedback, written as

$$\mathcal{G}_t = f_{\text{proj}}(\mathcal{G}_{t-1}, I_t, \tau_t), \quad \Delta_t = f_{\text{diff}}(\mathcal{G}_{t-1}, \mathcal{G}_t), \quad F_t = f_{\text{sum}}^B(\mathcal{G}_t, I_t, \Delta_t), \quad (1)$$

where  $f_{\text{proj}}$  is the trace projection that registers the intent and writes the execution into the graph,  $f_{\text{diff}}$  extracts the effect of the round, and  $f_{\text{sum}}$  organizes that effect into decision evidence under a feedback budget  $B$ . The agent then produces the next intent and program as  $(I_{t+1}, P_{t+1}) =$

$\pi(q, \mathcal{H}_t, F_t)$ , where  $\pi$  denotes the backbone LLM and  $\mathcal{H}_t$  holds the task instruction with the programs the agent itself wrote, so the raw outputs of past blocks stay in the runtime and reach the model only through  $F_t$ .

### 3.1 GRAPH-BASED EXECUTION MODELING

This stage turns a time-ordered running record into a typed execution graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  maintained per task, so that result provenance (Herschel et al., 2017), cross-block consumption, and state evolution obtain one representation. The node set  $\mathcal{V}$  covers program blocks, tool calls, artifacts, state versions, and failures, together with a task node and intent nodes, and the edge set  $\mathcal{E}$  carries typed relations for execution, data flow, and state (Appendix Table 4 lists the full type inventory).

**Typed Trace Projection.** The first structural change is that a code execution is modeled by its internal effects and not by its output string. During execution, tool wrappers record the arguments, results, and effects of every call, and  $f_{\text{proj}}$  attaches these calls to the current block  $b_t$  and supplements block outputs, state accesses, and failures, adding the edges

$$\{b_t \xrightarrow{\text{executes}} c \mid c \in \mathcal{C}_t\} \cup \{c \xrightarrow{\text{produces}} v \mid v \in \mathcal{A}_t\} \cup \{u_\phi \xrightarrow{\text{fails.at}} \phi \mid \phi \in \Phi_t\}, \quad (2)$$

where  $\mathcal{C}_t$  is the set of calls issued by  $b_t$ ,  $\mathcal{A}_t$  the artifacts they produce,  $\Phi_t$  the failures, and  $u_\phi$  the failing call when the failure is attributable to one call and the block  $b_t$  otherwise, and every call  $c$  carries a typed effect  $\varepsilon(c) \in \{\text{read}, \text{write}\}$  with success and state-change flags. Because calls inside loops and conditionals all enter the graph, multi-tool execution within one block becomes analyzable.

**Cross-Block Dependency Linking.** The second structural change is that history across blocks is a multi-relation graph and not a linear order. Guided by tool effect contracts,  $f_{\text{proj}}$  assigns each artifact a normalized identity key  $\kappa(\cdot)$  over its content, and links a later artifact to an earlier one when their keys agree,

$$v \xrightarrow{\text{equivalent.to}} v' \quad \text{when} \quad \kappa(v) = \kappa(v'), \quad (3)$$

so two differently phrased queries that return the same content are recognized as equivalent and not as distinct results. Equivalence is conservative and applies only to read-effect calls that the contract marks deterministic, so a write is never treated as a duplicate and a real state change is never masked. Subsequent use of an existing artifact is recorded as consumes when it serves as input and as reuses when execution reuses it directly, and observable state is versioned with supersedes edges between successive writes, so a later call links to an earlier write under the representation that also carries artifact dependencies. The output is the accumulated graph  $\mathcal{G}_t$  holding the current block with its calls, artifacts, states, failures, and cross-block links.

### 3.2 GRAPH-BASED PROGRESS ASSESSMENT

This stage relates what a block intended to do to what it actually changed, so that new results, repeated results, and target-local stagnation are separated.

**Intent Registration and Binding.** Here the purpose of a block becomes a first-class object bound to its observed effect. When the agent submits a program it declares an intent

$$I_t = (a_t, z_t, e_t), \quad (4)$$

where  $a_t$  is the action type,  $z_t$  the target node, and  $e_t$  the expected change, and the target may be the task node or an existing subgoal. Before execution  $f_{\text{proj}}$  registers an intent node with a targets edge to  $z_t$  and snapshots the current artifacts and states, and after execution it binds the intent to the executed block through an implemented\_by edge and attaches an action receipt, so later analysis can locate which attempt produced which result.

**Graph Delta and Stagnation Analysis.** Progress is defined on graph effects instead of on new output text. From the executed block,  $f_{\text{diff}}$  collects the produced artifacts  $\mathcal{A}_t$ , assigns to  $\mathcal{E}_t$  those carrying an equivalent\_to in-edge, and defines the new artifacts and the progress indicator as

$$\mathcal{N}_t = \mathcal{A}_t \setminus \mathcal{E}_t, \quad p_t = \mathbb{1}[\mathcal{N}_t \neq \emptyset \vee \mathcal{S}_t \neq \emptyset], \quad (5)$$

---

**Algorithm 1** GRAPHPTC performs graph-guided programmatic tool calling.

---

**Require:** task  $q$ , tools, feedback budget  $B$

```

1: initialize graph  $\mathcal{G} \leftarrow \{\text{task node}\}$ , feedback  $F \leftarrow \emptyset$ 
2: while budget remains do
3:    $(I, P) \leftarrow \pi(q, \mathcal{H}, F)$  {declare intent  $I = (a, z, e)$  and program}
4:   if  $a$  is ANSWER then
5:     return the answer
6:   end if
7:   register  $I$  and snapshot  $\mathcal{G}^- \leftarrow \mathcal{G}$ 
8:    $\tau \leftarrow$  execute  $P$  with instrumented tools
9:    $\mathcal{G} \leftarrow f_{\text{proj}}(\mathcal{G}, I, \tau)$  {typed projection and cross-block linking, Sec. 3.1}
10:   $\Delta \leftarrow f_{\text{diff}}(\mathcal{G}^-, \mathcal{G})$  over new artifacts  $\mathcal{N}$ , state effects  $\mathcal{S}$ , failures, stagnation {Sec. 3.2}
11:   $F \leftarrow f_{\text{sum}}^B(\mathcal{G}, I, \Delta)$  with opportunity set  $\mathcal{O} = f_{\text{opp}}(\Delta)$  {Sec. 3.3}
12: end while

```

---

where  $\mathcal{S}_t$  is the set of state effects linked by writes or mutates, so allocating a new result node or reusing a cached one does not count as new progress. A target-local stagnation count then follows the recurrence

$$s_t(z_t) = \begin{cases} 0, & p_t = 1, \\ s_{t-1}(z_t) + 1, & p_t = 0, \end{cases} \quad (6)$$

where counts on other targets stay independent, so the agent is told when a target stops advancing even though no error is raised. The count is advisory and the agent may override it, since a plan change is only suggested once the count passes a threshold. The output is the round effect  $\Delta_t$ , the receipt bound to the declared action, and the stagnation count.

### 3.3 GRAPH-BASED ACTION SELECTION

This stage turns the graph evidence into a bounded feedback and offers the next action with its reason, and the agent makes the choice.

**Bounded Feedback Construction.** The agent receives an evidence-focused view of the graph instead of the whole graph. From the accumulated graph, the intent, and the delta,  $f_{\text{sum}}$  extracts the declared action, its receipt, the new and equivalent artifacts, the state changes, the failure context, and the prior attempts on the target, and reduces them under the budget by pruning list items and then fields, so that  $|F_t| \leq B$  and the long-horizon record stays at runtime.

**Graph-Guided Action Selection.** Recovery is exposed as an addressable graph operation instead of free-form retrying. The runtime turns execution signals into an opportunity set  $\mathcal{O}_t = f_{\text{opp}}(\Delta_t)$ , where a failure yields a repair opportunity and a stagnation that reaches a threshold yields a replanning opportunity, and the agent selects an action

$$a_{t+1} \in \{\text{CONTINUE}, \text{PATCH}, \text{REPLAN}, \text{ANSWER}\}, \quad (7)$$

where CONTINUE advances a new step, PATCH returns to a failed node and re-executes, REPLAN changes the dependency path, and ANSWER terminates. The first three actions carry a new intent and program and re-enter Eq. 1, so an action label is tied to concrete program behavior. Algorithm 1 summarizes the loop.

## 4 EXPERIMENTS

### 4.1 BENCHMARKS AND SETUP

We evaluate on seven tool-use benchmark settings that span retrieval, question answering, application control, and workflow automation. We choose these benchmarks because they stress the cross-block dependency that GraphPTC targets along complementary axes, from short to long horizons, and each carries an executable, rule-based, or judge-based evaluation against a reference. **BrowseComp-Plus** (Chen et al., 2025) measures evidence-seeking browsing over a fixed corpus

Table 1: Main results on seven tool-use benchmark settings across five backbones. Each column reports the primary metric of one setting, with answer accuracy for BrowseComp-Plus, Task Goal Completion (TGC) for the two AppWorld splits, pass rate for Agent-Diff and AutomationBench, and judge accuracy for FanOutQA and FRAMES. **Bold** marks the best method per backbone and benchmark. GraphPTC improves over both Direct Tool Calling and PTC on every benchmark.

| Method                 | BrowseComp-Plus<br>(Acc) | AppWorld-Normal<br>(TGC) | AppWorld-Challenge<br>(TGC) | Agent-Diff<br>(Pass) | FanOutQA<br>(Acc) | FRAMES<br>(Acc) | Automation-Bench<br>(Pass) |
|------------------------|--------------------------|--------------------------|-----------------------------|----------------------|-------------------|-----------------|----------------------------|
| <i>GPT-5.6-Sol</i>     |                          |                          |                             |                      |                   |                 |                            |
| Direct Tool Calling    | 73.49                    | 82.10                    | 81.80                       | 51.34                | 55.81             | 77.31           | 20.83                      |
| PTC                    | 80.00                    | 89.90                    | 90.60                       | 39.73                | 52.58             | 73.42           | 29.17                      |
| <b>GraphPTC</b>        | <b>80.60</b>             | <b>91.70</b>             | <b>93.00</b>                | <b>51.49</b>         | <b>60.00</b>      | <b>79.00</b>    | <b>31.00</b>               |
| <i>Claude-Opus-5</i>   |                          |                          |                             |                      |                   |                 |                            |
| Direct Tool Calling    | 43.98                    | 94.00                    | 80.80                       | 75.14                | 77.74             | 84.95           | 28.67                      |
| PTC                    | 10.96                    | 59.50                    | 79.10                       | 68.06                | 76.77             | 85.19           | 32.50                      |
| <b>GraphPTC</b>        | <b>49.04</b>             | <b>95.80</b>             | <b>95.00</b>                | <b>79.17</b>         | <b>78.71</b>      | <b>85.56</b>    | <b>33.50</b>               |
| <i>DeepSeek-V4-Pro</i> |                          |                          |                             |                      |                   |                 |                            |
| Direct Tool Calling    | 4.58                     | 81.00                    | 65.70                       | 70.09                | 57.42             | 81.43           | 21.50                      |
| PTC                    | 3.49                     | 88.70                    | 80.10                       | 72.47                | 58.06             | 80.83           | 17.00                      |
| <b>GraphPTC</b>        | <b>13.37</b>             | <b>89.90</b>             | <b>81.50</b>                | <b>73.96</b>         | <b>62.26</b>      | <b>81.55</b>    | <b>22.83</b>               |
| <i>Kimi-K3</i>         |                          |                          |                             |                      |                   |                 |                            |
| Direct Tool Calling    | 8.92                     | 72.00                    | 46.50                       | 77.53                | 49.03             | 73.54           | 27.50                      |
| PTC                    | 2.89                     | 85.70                    | 83.20                       | 78.12                | 28.71             | 75.85           | 28.00                      |
| <b>GraphPTC</b>        | <b>10.48</b>             | <b>86.30</b>             | <b>84.90</b>                | <b>78.72</b>         | <b>50.65</b>      | <b>76.46</b>    | <b>28.83</b>               |
| <i>Qwen3.8-Max</i>     |                          |                          |                             |                      |                   |                 |                            |
| Direct Tool Calling    | 34.46                    | 82.70                    | 71.70                       | 74.26                | 65.81             | 79.98           | 31.67                      |
| PTC                    | 39.76                    | 95.20                    | 85.40                       | 78.12                | 59.68             | 80.10           | 32.38                      |
| <b>GraphPTC</b>        | <b>41.93</b>             | <b>97.00</b>             | <b>87.50</b>                | <b>80.06</b>         | <b>66.77</b>      | <b>82.40</b>    | <b>42.41</b>               |

and reports answer accuracy. **AppWorld** (Trivedi et al., 2024) is an executable environment of everyday apps, and we use its normal and challenge splits scored by Task Goal Completion (TGC) and Scenario Goal Completion (SGC). **Agent-Diff** (Pysklo et al., 2026) measures stateful multi-step tasks with an assertion-based pass rate. **FanOutQA** (Zhu et al., 2024) measures multi-hop, multi-document question answering with a judge accuracy. **FRAMES** (Krishna et al., 2025) measures fact retrieval and reasoning with a judge accuracy. **AutomationBench** (Shepard & Salimans, 2026) measures business workflow automation with a pass rate over end-to-end workflows. The two AppWorld splits give seven evaluation settings in total, and we run every setting with all five backbones.

We instantiate three methods under one shared protocol. *Direct Tool Calling* uses native per-call function calling, *PTC* uses programmatic tool calling with a persistent runtime and no graph, and *GraphPTC* adds the dynamic execution graph of Section 3. All three share the same tools, prompts, and budgets on each benchmark, and the opportunity rules and feedback budget of GraphPTC are held fixed across all benchmarks and backbones, so a gain reflects the graph and not per-setting tuning. The five backbones, GPT-5.6-Sol, Claude-Opus-5, DeepSeek-V4-Pro, Kimi-K3, and Qwen3.8-Max, cover closed and open, dense and mixture-of-experts models.

## 4.2 MAIN RESULTS

**Capability on Application Tool Calling.** We evaluate GRAPHPTC on the AppWorld normal and challenge splits to assess its *multi-step application control*, where a task drives many app calls whose write effects must persist and compose into a final goal. As shown in Table 1, GRAPHPTC improves over both Direct Tool Calling and PTC on both splits for every backbone, and its five-backbone average gain over Direct Tool Calling grows from 9.8 points on the normal split to 19.1 points on the challenge split. We attribute this pattern to modeling state as typed nodes, so a later block links to an earlier write instead of re-deriving it from text, and Section 5 tests this attribution directly.

**Capability on Multi-Hop Tool Calling.** We evaluate GRAPHPTC on BrowseComp-Plus, FanOutQA, and FRAMES to assess its *multi-hop retrieval and reasoning*, where an answer depends on composing evidence gathered across many calls. As shown in Table 1, GRAPHPTC improves over both baselines on all three benchmarks for every backbone, and it lifts the collapsed PTC accuracy of Claude-Opus-5 on BrowseComp-Plus from 10.96 to 49.04, above the 43.98 of Direct Tool

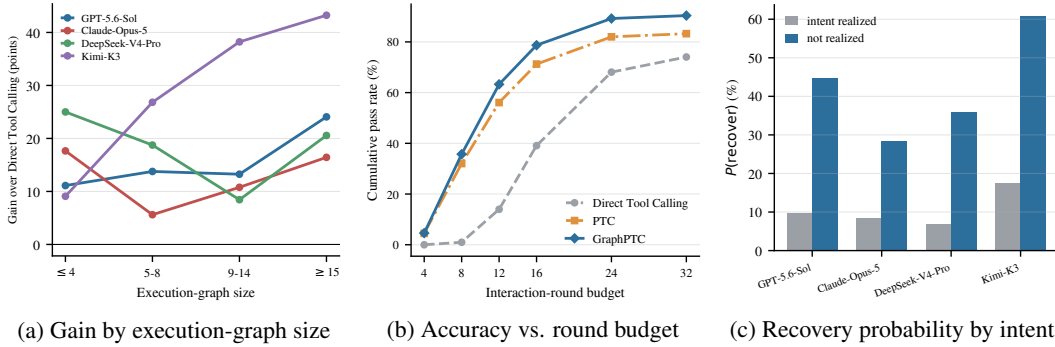


Figure 2: Analyses of GraphPTC on AppWorld, averaged over four backbones. (a) The gain over Direct Tool Calling is positive in every complexity bin, and its average widens toward the most complex tasks. (b) Along the accuracy against interaction-round frontier GraphPTC reaches the accuracy of Direct Tool Calling with fewer rounds. (c) An intent that is not realized raises the probability that the agent issues a recovery action.

Calling. This recovery reflects cross-block dependency linking, which marks equivalent results and lets the agent stop re-issuing queries whose content it already holds.

**Capability on Long-Horizon Tool Calling.** We evaluate GRAPHPTC on Agent-Diff and AutomationBench to assess its *stateful workflow automation*, where a task completes only when a chain of dependent write operations all take effect. As shown in Table 1, GRAPHPTC improves over both baselines on both benchmarks for every backbone, and on AutomationBench it turns the DeepSeek-V4-Pro case where PTC loses 4.5 points to Direct Tool Calling into a gain of 1.3 points. Here progress assessment separates a genuine state change from a re-executed no-op, so the agent advances the workflow instead of restating a completed step.

Across the panel, GRAPHPTC is the best method on every one of the seven settings for all five backbones, improving over PTC by 6.3 points and over Direct Tool Calling by 6.8 points on average. PTC itself falls below Direct Tool Calling 14 times over the same settings and backbones, losing 34.5 points on Claude-Opus-5 in the AppWorld normal split, so a programmatic runtime needs structure to pay off and is not a uniform upgrade.

## 5 ANALYSES

We conduct in-depth analyses to understand where the advantage of GraphPTC comes from and how the execution graph changes tool-calling behavior.

### 5.1 ABLATION STUDY ON GRAPH MODELING

To locate the contribution of the graph layer, we ablate the two structural changes and report the five-backbone average of each setting. Removing the execution graph and its graph-guided recovery reduces GraphPTC to PTC, where a block reports a plain success flag with raw output instead of typed effects, and removing the programmatic runtime as well reduces it to Direct Tool Calling. As shown in Table 2, the execution graph is worth 6.3 points on average and stays positive on all seven settings, while the programmatic runtime by itself is worth only 0.5 points and turns negative on four of them, so writing code by itself does not reliably improve on Direct Tool Calling and the graph is what turns it into a consistent gain.

The size of the graph contribution tracks how much cross-block dependency a setting carries, reaching 11.7 points on BrowseComp-Plus and 8.3 points on AppWorld-Normal, where a result produced in one block is consumed by a later block, and falling to 1.9 points on FRAMES, where most questions resolve within a few independent lookups. The loss of plain PTC concentrates in the same settings and on one backbone, since its worst cells are -33.0 points on BrowseComp-Plus and -34.5 points on AppWorld-Normal, both on Claude-Opus-5, which loses 10.5 points under plain PTC while the other four backbones gain from it.

Table 2: Ablation of the two structural changes of GraphPTC, reported per setting as the five-backbone average. PTC removes the execution graph and Direct Tool Calling removes the programmatic runtime as well, and the last two rows split the total gap into the contribution of each change.

| Configuration                            | BrowseComp-Plus | AppWorld-Normal | AppWorld-Challenge | Agent-Diff  | FanOutQA    | FRAMES      | Automation-Bench | Avg         |
|------------------------------------------|-----------------|-----------------|--------------------|-------------|-------------|-------------|------------------|-------------|
| <b>GraphPTC (full)</b>                   | <b>39.1</b>     | <b>92.1</b>     | <b>88.4</b>        | <b>72.7</b> | <b>63.7</b> | <b>81.0</b> | <b>31.7</b>      | <b>67.0</b> |
| – execution graph                        | 27.4            | 83.8            | 83.7               | 67.3        | 55.2        | 79.1        | 27.8             | 60.6        |
| – execution graph and programs           | 33.1            | 82.4            | 69.3               | 69.7        | 61.2        | 79.4        | 26.0             | 60.2        |
| Contribution of the execution graph      | +11.7           | +8.3            | +4.7               | +5.4        | +8.5        | +1.9        | +3.9             | +6.3        |
| Contribution of the programmatic runtime | −5.7            | +1.4            | +14.4              | −2.4        | −6.0        | −0.4        | +1.8             | +0.5        |

## 5.2 ADVANTAGE OF NON-LINEAR RECOVERY

To test whether modeling cross-block dependency as a graph enables recovery a linear history cannot, we study how the agent uses the actions that return to a specific node. We measure the probability that the agent issues a recovery action conditioned on whether the previous block realized its declared intent, and on the tasks where GraphPTC uses recovery we compare its pass rate with PTC on the same task identifiers. As shown in Figure 2c, an unrealized intent raises the recovery probability by three to five times across backbones, so the declared-and-verified intent is an actionable signal that the agent responds to. On the same task identifiers GraphPTC exceeds PTC, and since the comparison fixes the task and differs mainly in whether the agent can return to a failed node, we read the gap as largely driven by non-linear recovery.

## 5.3 ADVANTAGE ON COMPLEX LONG-HORIZON TASKS

To locate where the advantage of GraphPTC arises, we bin every AppWorld task by the number of program blocks in its execution graph and measure the gain of GraphPTC over Direct Tool Calling within each bin. As shown in Figure 2a, the gain is positive in every bin for every backbone, and its four-backbone average rises from 15.7 points in the smallest bin to 26.1 points in the largest. The widening is carried by GPT-5.6-Sol and Kimi-K3, which reach 24.1 and 43.2 points in the largest bin, while Claude-Opus-5 and DeepSeek-V4-Pro gain in every bin without a monotone trend. A larger task graph holds more intermediate results and more failure-prone steps, so addressable artifacts and local recovery matter more as a linear history grows harder to reconstruct.

## 5.4 EFFICIENCY AND TOKEN ECONOMY

To check whether the graph improves efficiency and not only accuracy, we measure the interaction rounds each method spends and trace the accuracy it reaches under a round budget. On Automation-Bench, GraphPTC spends 54 to 64% fewer model rounds than Direct Tool Calling while raising the pass rate, because one block carries several dependent calls whose intermediate values are registered as artifacts and reused (Appendix C). As shown in Figure 2b, on AppWorld the same effect appears along the accuracy against round frontier, where GraphPTC reaches the accuracy of Direct Tool Calling with about 2 times fewer rounds and stays on or above the PTC frontier at every budget. The graph stays in the runtime and enters the model only through the bounded feedback, so the input per round is independent of graph size and the round savings are not offset by a growing context. As shown in Table 3, GraphPTC reaches higher accuracy than Direct Tool Calling on every benchmark while reading 1.7 to 2.3 times fewer input tokens, and uses fewer rounds on all but one benchmark, since a per-call transcript re-sends the growing history on each turn.

## 5.5 ROBUSTNESS ACROSS BACKBONE CAPABILITY

To examine how the benefit depends on backbone capability, we group the five backbones into frontier ones, GPT-5.6-Sol and Claude-Opus-5, and second-tier ones, DeepSeek-V4-Pro, Kimi-K3, and Qwen3.8-Max, and compare the gain over Direct Tool Calling. As shown in Figure 3a, the second-tier backbones gain more than the frontier ones. This follows because the execution graph externalizes the state tracking and dependency bookkeeping that a frontier model partly carries on its own, so a second-tier model benefits more when that structure is supplied. At the same time, the graph is what keeps PTC from collapsing on a frontier backbone, since Claude-Opus-5 loses accuracy under plain PTC yet reaches the best result under GraphPTC.



Table 3: Cost comparison of GraphPTC and Direct Tool Calling, averaged over four backbones. GraphPTC reaches higher accuracy while reading fewer input tokens on every benchmark and using fewer interaction rounds on all but one. **Bold** marks the better method per metric and benchmark.

| Benchmark          | Accuracy (%)        |             | Input tokens (k)    |              | Rounds              |             |
|--------------------|---------------------|-------------|---------------------|--------------|---------------------|-------------|
|                    | Direct Tool Calling | GraphPTC    | Direct Tool Calling | GraphPTC     | Direct Tool Calling | GraphPTC    |
| AppWorld-Normal    | 82.3                | <b>90.9</b> | 151.4               | <b>84.1</b>  | 16.4                | <b>9.1</b>  |
| AppWorld-Challenge | 68.7                | <b>88.6</b> | 228.8               | <b>133.5</b> | 18.7                | <b>11.7</b> |
| Agent-Diff         | 68.5                | <b>70.8</b> | 113.2               | <b>49.0</b>  | 9.6                 | <b>6.6</b>  |
| FanOutQA           | 60.0                | <b>62.9</b> | 128.5               | <b>63.9</b>  | 8.7                 | <b>6.9</b>  |
| FRAMES             | 79.3                | <b>80.6</b> | 93.0                | <b>55.2</b>  | <b>5.6</b>          | 7.1         |
| AutomationBench    | 24.6                | <b>29.0</b> | 324.4               | <b>171.9</b> | 36.1                | <b>14.3</b> |

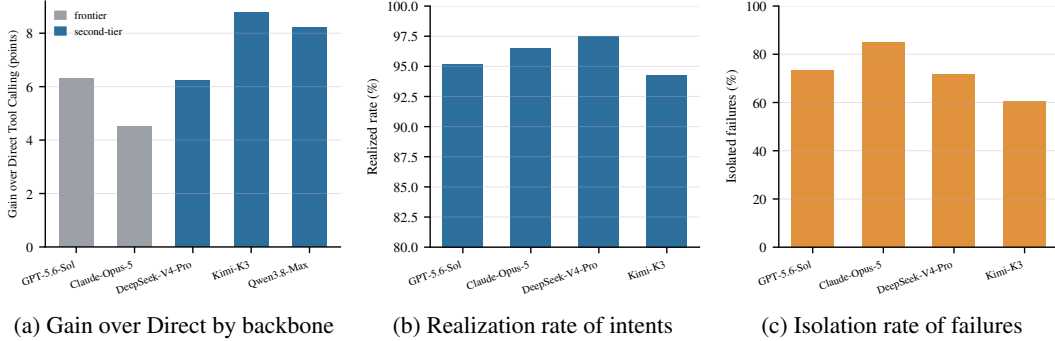


Figure 3: Further analyses of GraphPTC. (a) Gain over Direct Tool Calling by backbone on the seven-setting average; the second-tier backbones gain more from the graph than the frontier ones. (b) On AppWorld over four backbones, most declared expected changes are realized in the graph. (c) On AppWorld over four backbones, most block failures are isolated, with no further failure within three blocks.

## 5.6 RELIABILITY OF THE INTENT SIGNAL

To assess whether the progress signal the agent acts on is trustworthy, we measure how often a declared expected change is realized in the graph and how often a block failure stays local. As shown in Figure 3b, the declared intent is realized in more than nine of ten blocks across backbones, so the signal that drives recovery is grounded in observed effects and not in self-report. As shown in Figure 3c, most block failures under GraphPTC are isolated with no further failure in the next three blocks, while under plain PTC a failing task tends to repeat the same exception.

## 5.7 LIMITATIONS

Our study is inference-time and adds no training, so a backbone that cannot follow the intent declaration gains less from the graph. We report one run per setting and treat the agreement of the trend across five backbones and seven settings as the evidence of robustness. The controlled comparison holds tools, prompts, and budgets fixed, so it isolates the graph but does not combine GraphPTC with reflection or long-term memory, which could be layered on the same graph.

## 6 CONCLUSION

In this paper, we propose GRAPHPTC, which models programmatic tool calling as a task-level dynamic execution graph. Specifically, GRAPHPTC turns the linear execution history of PTC into a typed structure over in-block tool effects and cross-block dependencies, so the agent reads what an execution changed and chooses to continue, patch, replan, or answer. Across seven benchmark settings and five backbones, GRAPHPTC is the best method in every case, improving over PTC by 6.3 points and over Direct Tool Calling by 6.8 points while reading 1.7 to 2.3 times fewer input tokens. Indexing what an execution did and not only what an agent saw is a broadly useful design pattern for agents that act on their own history.

## REPRODUCIBILITY STATEMENT

The method is specified in Section 3 and Algorithm 1, with node and edge types in Appendix A and further details in Appendix B. All arms share one protocol described in Section 4.1, and the plotting code and the raw numbers behind every analysis figure are released under the `code/` directory.

## AI USE STATEMENT

Generative AI tools were used to assist with language editing of the manuscript. All technical content, experiments, and analyses were designed, run, and verified by the authors, who take responsibility for the final content.

## REFERENCES

- Anthropic. Code execution with MCP: Building more efficient agents. <https://www.anthropic.com/engineering/code-execution-with-mcp>, 2025.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.
- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhu Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. *arXiv preprint arXiv:2508.06600*, 2025.
- Susan B. Davidson and Juliana Freire. Provenance and scientific workflows: challenges and opportunities. In *ACM SIGMOD International Conference on Management of Data*, pp. 1345–1350, 2008.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pp. 10764–10799, 2023.
- Melanie Herschel, Ralf Diestelkämper, and Houssem Ben Lahmar. A survey on provenance: What for? what form? what from? *The VLDB Journal*, 26(6):881–906, 2017.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- Hugging Face. smolagents: A barebones library for agents that think in code. <https://github.com/huggingface/smolagents>, 2025.
- Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqui. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In *Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. In *arXiv preprint arXiv:2310.08560*, 2023.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37, 2024.

- Hubert M. Pysklo, Artem Zhuravel, and Patrick D. Watson. Agent-diff: Benchmarking llm agents on enterprise api tasks via code execution with state-diff-based evaluation. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2026.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, 2024.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.
- Daniel Shepard and Robin Salimans. Automationbench: Evaluating ai agents on cross-application workflow orchestration. *arXiv preprint arXiv:2604.18934*, 2026.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Annual Meeting of the Association for Computational Linguistics*, 2024.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *International Conference on Machine Learning*, 2024.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. In *Advances in Neural Information Processing Systems*, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Andrew Zhu, Alyssa Hwang, Liam Dugan, and Chris Callison-Burch. Fanoutqa: A multi-hop, multi-document question answering benchmark for large language models. *Annual Meeting of the Association for Computational Linguistics*, 2024.

## A NODE AND EDGE TYPES OF THE EXECUTION GRAPH

Table 4 lists the typed nodes and edges maintained in the execution graph. Node types cover the task, program blocks, tool or API calls with read or write effects, artifacts, state versions, failures, and action intents. Edge types cover execution structure, data flow, state evolution, failure attribution, cross-block equivalence and reuse, and intent binding.

Table 4: Node and edge types of the dynamic execution graph.

| Category    | Types                                                            |
|-------------|------------------------------------------------------------------|
| Nodes       | TASK, BLOCK, CALL (read/write), ARTIFACT, STATE, FAILURE, INTENT |
| Execution   | contains, executes, produces, fails_at                           |
| Data flow   | consumes, reuses, derives                                        |
| State       | reads, writes, mutates, supersedes                               |
| Cross-block | equivalent_to                                                    |
| Intent      | targets, implemented_by                                          |

## B IMPLEMENTATION DETAILS

The execution graph is maintained incrementally with node and edge cursors, so extracting the round delta  $\Delta_t$  visits only the nodes and edges added in the current round and does not solve a general graph edit distance. Tool wrappers record arguments, results, and effects during execution, and the trace projector attaches these to the current block after execution. Cross-block equivalence uses tool effect contracts to normalize argument and result identities, so cached deterministic reads may be reused while tools with side effects follow their contract. The bounded feedback prunes list items first and then reduces fields under the character budget  $B$ , and the full graph is retained at runtime and never sent to the model in whole.

## C ADDITIONAL ANALYSES

**Scenario Goal Completion on AppWorld.** Figure 4 reports the secondary AppWorld metric, Scenario Goal Completion, which requires every task in a scenario to succeed together. GraphPTC improves Scenario Goal Completion on both splits for every backbone, and the margin over PTC is wider than on Task Goal Completion, which matches the intuition that a scenario is only as strong as its most dependent step.

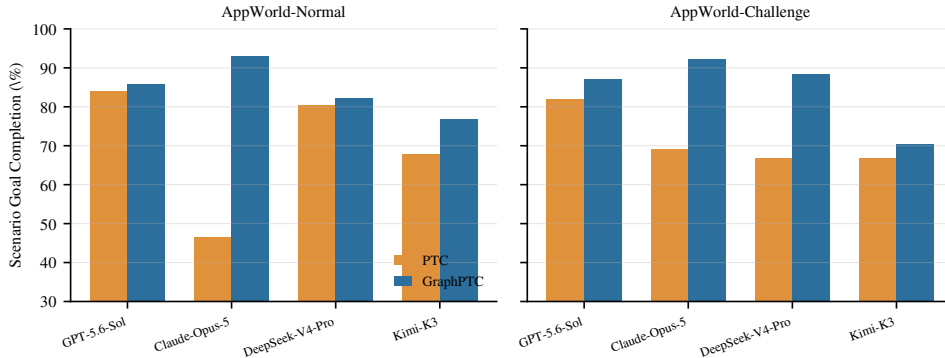


Figure 4: Scenario Goal Completion on AppWorld. GraphPTC beats PTC on every backbone and split, and the margin is wider than on Task Goal Completion.

**Use of the recovery actions.** Table 5 counts how often the agent issues PATCH and REPLAN on AppWorld and the pass rate of the tasks that use them. The recovery actions are used on a large share of tasks, and the tasks that trigger a PATCH still pass in almost all cases, which shows the graph turns a failure into a targeted retry that usually succeeds.

**Interaction rounds on AutomationBench.** Figure 5 reports the average model rounds per task on AutomationBench. GraphPTC spends 54 to 64% fewer rounds than Direct Tool Calling on every backbone while reaching a higher pass rate, which complements the AppWorld frontier in the main text and shows the efficiency gain is not specific to one benchmark.

**Input tokens across benchmarks.** Figure 6 reports the mean input tokens a task consumes, averaged over four backbones, for six of the seven settings. On every benchmark GraphPTC reads far fewer input tokens than Direct Tool Calling and between 1.1 and 1.4 times more than PTC, so the

Table 5: Use of the recovery actions on AppWorld, summed over the normal and challenge splits. Tasks that trigger a PATCH pass in almost all cases.

| Backbone        | PATCH calls | REPLAN calls | Pass rate on PATCH tasks |
|-----------------|-------------|--------------|--------------------------|
| GPT-5.6-Sol     | 516         | 52           | 100.0                    |
| Claude-Opus-5   | 506         | 20           | 97.8                     |
| DeepSeek-V4-Pro | 586         | 0            | 94.7                     |
| Kimi-K3         | 972         | 3            | 99.1                     |

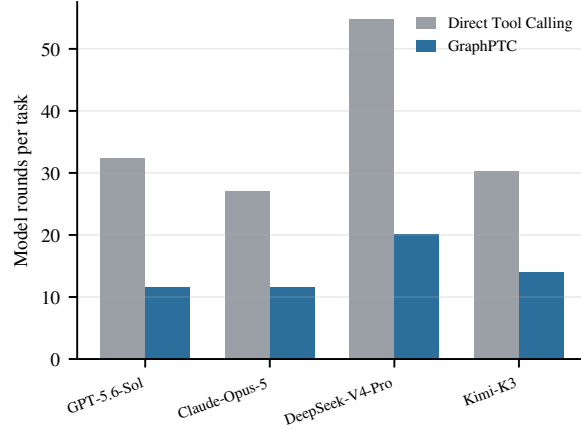


Figure 5: Average model rounds per task on AutomationBench. GraphPTC reaches a higher pass rate with far fewer rounds on every backbone.

token economy shown for AppWorld in the main text holds throughout, since a per-call transcript re-sends the growing history on each turn while the bounded feedback does not.

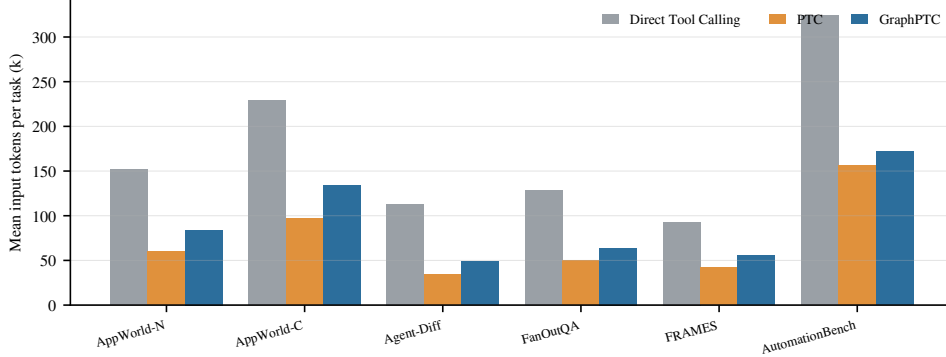


Figure 6: Mean input tokens per task, averaged over four backbones. GraphPTC reads far fewer input tokens than Direct Tool Calling on every benchmark while reaching the best accuracy.

**Gain by reasoning type on FRAMES.** Figure 7 breaks the FRAMES gain down by reasoning type for the two closed backbones. The gain is largest on post-processing and temporal reasoning, where an answer must combine results across several calls, which is where cross-block linking removes the most redundant work.

**Where the workflow gain arises.** Agent-Diff labels each task by the operations it performs, and Figure 8 splits the tasks by whether they update or delete existing state. GraphPTC improves over PTC by 7.2 points on the state-overwriting tasks and is flat on the tasks with no overwrite, which matches the mechanism, since the state versioning of the graph helps exactly where a later step depends on a mutated state and adds little where there is no state to track.

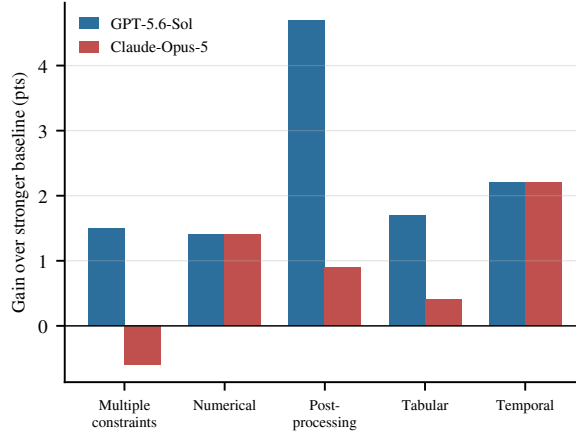


Figure 7: FRAMES gain of GraphPTC over the stronger baseline by reasoning type, in accuracy points. The gain concentrates on post-processing and temporal reasoning.

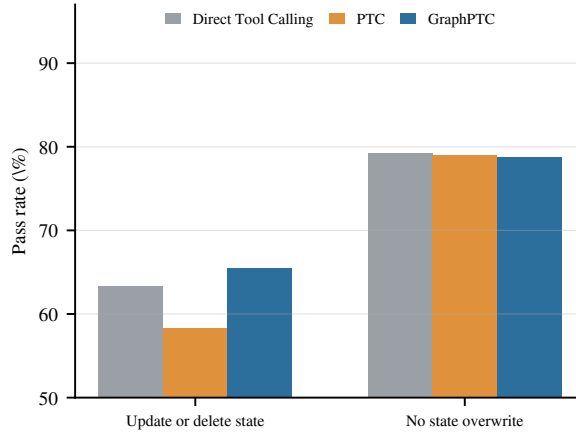


Figure 8: Agent-Diff pass rate split by whether a task overwrites existing state, aggregated over four backbones. GraphPTC’s gain concentrates on state-overwriting tasks.

## D CASE STUDIES

We give three traces that show where the execution graph changes what the agent can do. Each case draws an abbreviated but complete trace on the same task under two settings, with panel (a) showing a linear PTC history and panel (b) showing the GraphPTC execution graph, and each accompanying description walks through the flow step by step and points to the graph element that carries the decision.

**Case 1, cross-block equivalence removes redundant retrieval.** The task asks for the birthplace of the author of a specific novel, which is a two-hop lookup where the intermediate entity, the author’s name, drives the second query. As drawn in Figure 9, both settings begin by writing three blocks. Block  $P_1$  issues a search that resolves the author, block  $P_2$  consumes that intermediate to search the author’s birthplace, and block  $P_3$  issues a rephrased composite query, either because a self-check prompted a rewording or because the model wrote the composite lookup independently of the earlier step-by-step lookups. Under PTC the three outputs enter the history as three separate strings and their identity is not represented, so the agent cannot tell that the third string encodes the same fact as the second, and it opens a fourth block to reconcile the two answers. Under GraphPTC the trace projector assigns each search result an identity key  $\kappa(\cdot)$  computed from the normalized search target and the returned entity, and when the third result’s key matches the second the projector writes an `equivalent_to` edge  $v_3 \rightarrow v_2$ . The delta  $\Delta_3$  then reports an empty novel set and a progress indicator  $p_3 = 0$ , and the bounded feedback labels the third artifact as equivalent evidence

and not as a new result. Given this signal, the action selector chooses ANSWER on the third round, and the trace closes without a fourth block. The mechanism that separates the two behaviors is not the reasoning of the model, but the graph, which converts a duplicated string into a typed cross-block edge that is directly readable.

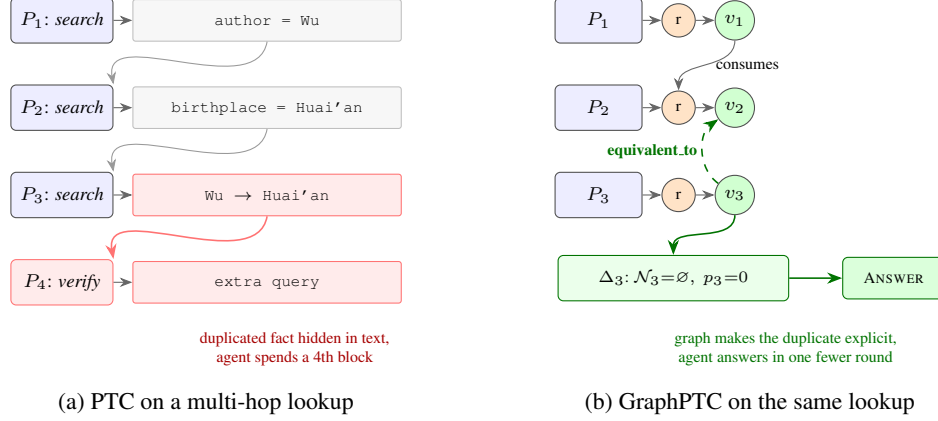


Figure 9: Case 1, cross-block equivalence on a multi-hop question. (a) A linear history records three raw output strings and hides that the third and second carry the same fact under different phrasings. (b) The execution graph gives the two results one identity key  $\kappa(\cdot)$  and one `equivalent.to` edge, and the delta  $\Delta_3$  carries the signal that no new artifact appeared.

**Case 2, a failed write becomes a targeted patch.** The task is an AppWorld workflow that fetches the latest phone bill, resolves the recipient’s email, and sends the bill as an attachment. As drawn in Figure 10, both settings issue three tool calls across blocks  $P_1$ ,  $P_2$ , and  $P_3$ , and the third call fails with a type error because the send argument expected a file path and received an integer id. Under PTC the failure appears as one line inside the long stdout of the third block, and the agent must recover from raw text, so it re-writes the whole block, which re-issues the two lookups that had already succeeded and adds a corrected send, a 4-th block that repeats two calls the runtime had already run. Under GraphPTC the projector attaches a `FAILURE` node with a `fails_at` edge from the specific call  $c_3$ , and the delta  $\Delta_3$  places that failure into the opportunity set. The action selector then declares  $I_4 = (\text{PATCH}, z = c_3, e = \text{success})$ , and because the intent’s target is bound to the failed call the runtime executes only  $c_3$  with the corrected argument, while  $v_1$  and  $v_2$  remain addressable artifacts that the send call consumes through their existing identifiers. The typed failure and the intent-target binding are graph structures that a linear history cannot supply, and they are precisely what lets the agent repair one call instead of re-running many.

**Case 3, a later read is routed to the current write.** The task is an AppWorld workflow that updates a contact’s phone number and then sends a confirmation code to that contact. As drawn in Figure 11, both settings begin with  $P_1$  reading the current phone,  $P_2$  writing the new phone, and  $P_3$  needing the phone to send the code. Under PTC the transcript now carries two values for the same field, the pre-update 555-1234 from  $P_1$  and the post-update 555-9999 from  $P_2$ , and the agent must pick one from text alone, so a reasonable but wrong choice sends the SMS to the stale number, which forces a fourth block that re-fetches the contact and resends. Under GraphPTC the trace projector treats the observable state as a versioned object, so  $P_1$ ’s read attaches to the state version  $v_{\text{old}}$ ,  $P_2$ ’s write produces a new state version  $v_{\text{new}}$  and adds a `supersedes` edge  $v_{\text{new}} \rightarrow v_{\text{old}}$ , and  $P_3$ ’s read is resolved by the runtime to  $v_{\text{new}}$  through the same edge. The send then goes to the current number on the first attempt, and the fourth block is not needed. The key element is the state versioning, since it lets the runtime, and not the model, guarantee that a later read reflects the most recent write, which a linear transcript can only offer as two juxtaposed strings.

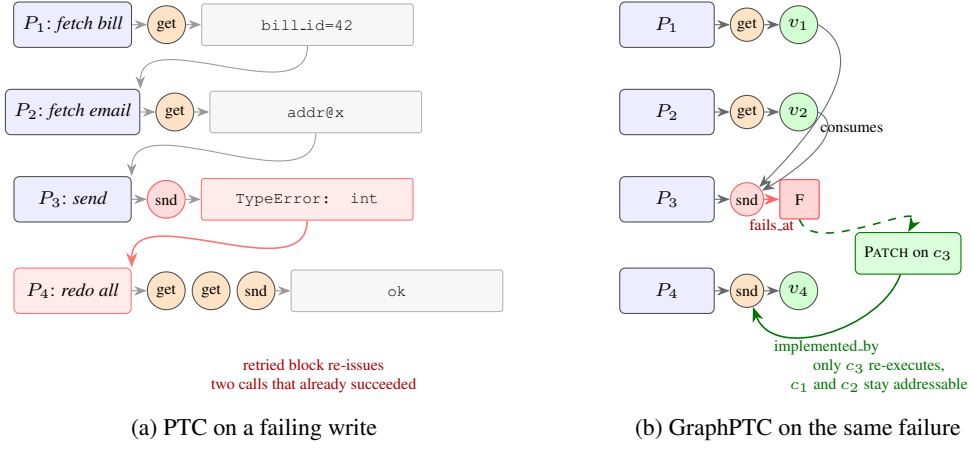


Figure 10: Case 2, a failed write on an AppWorld task. (a) A linear history places the failure inside a long output, so the agent re-writes the whole block and re-issues calls that already succeeded. (b) The execution graph binds the failure to a specific call through a `fails_at` edge, and a PATCH intent targets that call for local repair.

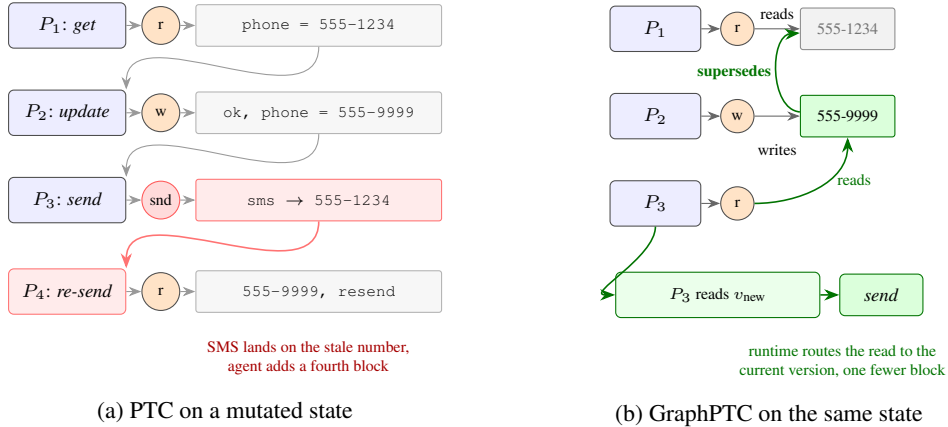


Figure 11: Case 3, stale state on a long-horizon workflow. (a) A linear history holds both the pre-update and post-update values as juxtaposed strings, and the agent can send to the wrong one. (b) The execution graph versions the observable state and a `supersedes` edge routes any later read to the current version.